# BÁO CÁO THỰC HÀNH OOP - TUẦN 5

**MSSV:** 202418834 **Họ tên:** Trần Tuấn Anh

Ngày 10 tháng 10 năm 2026

## 1 Link Github - Mã nguồn

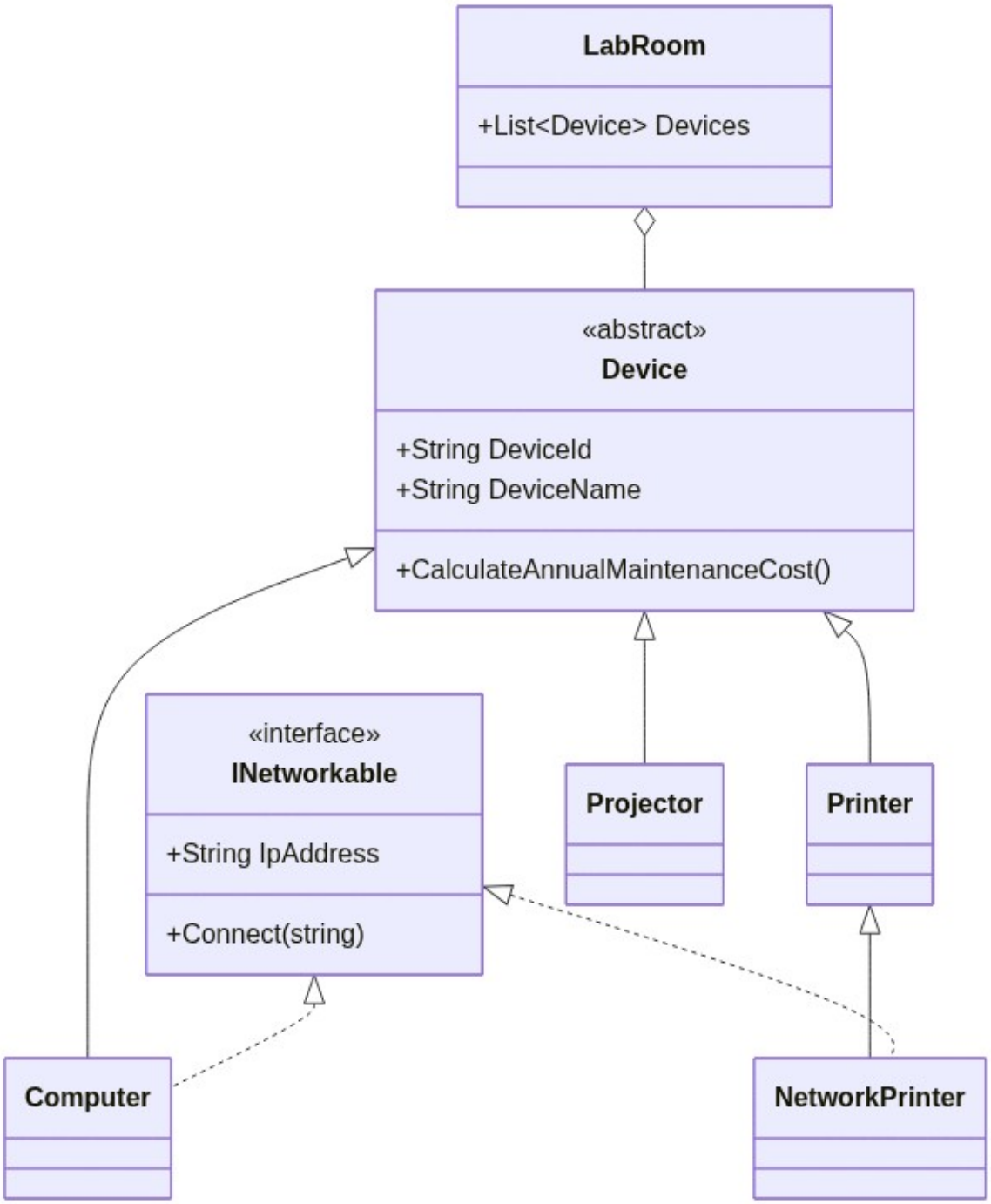
Link kho lưu trữ Github chứa mã nguồn: https://github.com/[YourUsername]/LabW05a

## 2 Phân tích yêu cầu nghiệp vụ

Hệ thống quản lý thiết bị và bảo trì trong phòng thực hành yêu cầu khả năng lưu trữ thông tin chung của các thiết bị (như mã, tên, giá, năm sử dụng) và tính toán chi phí bảo trì tùy theo từng loại thiết bị cụ thể.

- **Quản lý danh sách**: Hệ thống quản lý ba loại thiết bị chính (Máy tính, Máy in, Máy chiếu), mỗi loại có đặc thù riêng về thông số kỹ thuật (RAM/GPU, số trang in, số giờ sáng...).
- **Tính toán phí bảo trì**: Mỗi loại có một công thức tính chi phí bảo trì độc lập dựa trên hao mòn đặc trưng, đòi hỏi thiết kế đa hình.
- **Hỗ trợ mạng**: Có các thiết bị yêu cầu kết nối mạng (toàn bộ máy tính, một phần máy in), yêu cầu hệ thống cung cấp cách truy xuất đồng nhất đối với các thiết bị mạng mà không phụ thuộc vào lớp cụ thể của thiết bị.
- **Phân bổ**: Các thiết bị được phân bổ vào các phòng (LabRoom). Thiết bị có thể tồn tại độc lập với phòng (quan hệ Kết tập / Aggregation).

# 3 Danh sách lớp và sơ đồ quan hệ




*Sơ đồ lớp (Class Diagram) của hệ thống*

**Danh sách các lớp / giao diện:**

- `Device` (abstract class): Lớp cơ sở trừu tượng đại diện cho mọi thiết bị.
- `Computer`: Lớp dẫn xuất từ `Device`, thực thi `INetworkable`.
- `Printer`: Lớp dẫn xuất từ `Device`.
- `NetworkPrinter`: Lớp dẫn xuất từ `Printer`, thực thi `INetworkable` (Tách riêng để thiết kế mạng linh hoạt, thỏa mãn nguyên lý Liskov Substitution).
- `Projector`: Lớp dẫn xuất từ `Device`.

- `LabRoom`: Lớp đại diện phòng máy, chứa danh sách `Device`.
- `INetworkable` (interface): Giao diện mô tả khả năng kết nối mạng.
- `DeviceStatus`, `PrinterType` (enum): Trạng thái thiết bị và loại máy in.

**Quan hệ giữa LabRoom và Device:** Quan hệ giữa `LabRoom` và `Device` là quan hệ **Kết tập (Aggregation)**. Do theo nghiệp vụ, "Thiết bị vẫn có thể tồn tại trong hệ thống khi chưa được phân bổ vào phòng nào". Nếu phòng thực hành (`LabRoom`) bị xóa hoặc dỡ bỏ, các đối tượng `Device` vẫn tồn tại và có thể được phân bổ sang phòng khác, nghĩa là vòng đời của chúng hoàn toàn độc lập với nhau.

# 4 Kết quả chạy

## 1. Thêm thiết bị vào phòng

```
=== CHƯƠNG TRÌNH QUẢN LÝ THIẾT BỊ VÀ BẢO TRÌ ===
Thêm thiết bị thành công.
```

## 2. Thử thêm một thiết bị bị trùng mã

```
[Kiểm tra] Thêm thiết bị trùng mã:
-> Lỗi dự kiến: Thiết bị có mã C01 đã tồn tại trong phòng.
```

## 3. In danh sách thiết bị trong từng phòng

```
=== DANH SÁCH THIẾT BỊ ===
--- Phòng Máy 1 (Mã: R01) ---
[C01] Dell Optiplex - Năm SD: 2020, Giá: 15,000,000 VND, Trạng thái: Active | Máy tính: 16GB RAM, Core i5, GPU rời: False, Không có mạng
[C02] Alienware - Năm SD: 2023, Giá: 45,000,000 VND, Trạng thái: Active | Máy tính: 32GB RAM, Core i9, GPU rời: True, Không có mạng
[P01] Canon LBP2900 - Năm SD: 2021, Giá: 3,500,000 VND, Trạng thái: Active | Máy in: Laser, Đã in: 50000 trang, In màu: False
--- Phòng Máy 2 (Mã: R02) ---
[P02] Epson L805 - Năm SD: 2018, Giá: 6,500,000 VND, Trạng thái: UnderMaintenance | Máy in: Inkjet, Đã in: 120000 trang, In màu: True , Không có mạng (Hỗ
 trợ mạng)
[PR01] Sony VPL-EX - Năm SD: 2019, Giá: 12,000,000 VND, Trạng thái: Active | Máy chiếu: 3500 lumens, Số giờ bóng đèn: 3200h
```

## 4. Tính tổng chi phí bảo trì dự kiến của mỗi phòng

```
=== CHI PHÍ BẢO TRÌ DỰ KIẾN ===
Phòng Máy 1: 4,190,000 VND
Phòng Máy 2: 2,920,000 VND
```

### 5. Liệt kê thiết bị cần bảo trì

```
=== DANH SÁCH THIẾT BỊ CẦN BẢO TRÌ ===
Tại Phòng Máy 1:
 - Dell Optiplex (C01) | Lý do: Sử dụng > 5 năm
Tại Phòng Máy 2:
 - Epson L805 (P02) | Lý do: Đang bảo trì
 - Sony VPL-EX (PR01) | Lý do: Sử dụng > 5 năm
```

### 6. Kết nối mạng cho các đối tượng thực thi INetworkable

```
=== KẾT NỐI MẠNG ===
Đã kết nối mạng thành công cho các thiết bị hỗ trợ.
```

### 7. Duyệt các thiết bị mạng thông qua kiểu INetworkable, không phụ thuộc vào lớp cụ thể

```
=== DANH SÁCH THIẾT BỊ TRÊN MẠNG (INetworkable) ===
- Dell Optiplex đang kết nối tại IP: 192.168.1.10
- Alienware đang kết nối tại IP: 192.168.1.11
- Epson L805 đang kết nối tại IP: 192.168.1.50
```

# 5 Trả lời câu hỏi lý thuyết

## 5.1 1. Vì sao Device là lớp trừu tượng?

Lớp `Device` được thiết kế là lớp trừu tượng vì:

- **Không có ý nghĩa khi khởi tạo trực tiếp**: Trong thực tế, không có một "thiết bị chung chung"nào tồn tại, mà phải là một thiết bị cụ thể (Máy tính, Máy in...). Việc ngăn chặn khởi tạo đối tượng `Device` bằng từ khóa `abstract` giúp đảm bảo tính hợp lệ của dữ liệu.
- **Định nghĩa khung mẫu (Template)**: Nó đóng vai trò định nghĩa các thuộc tính và hành vi chung mà mọi thiết bị phải có. Đặc biệt, cung cấp phương thức `abstract CalculateAnnualMaintenanceCost()`, bắt buộc tất cả các lớp con phải tự cung cấp công thức tính phí bảo trì riêng biệt theo đặc thù của mình.

## 5.2 2. Vì sao INetworkable là interface?

`INetworkable` là interface vì:

- Khả năng kết nối mạng là một **năng lực (capability)** được bổ sung thêm chứ không phải là bản chất (không phải quan hệ "is-a"lõi). Do trong C# chỉ hỗ trợ đa kế thừa đối với interface, việc dùng interface cho phép một lớp như `Computer` vừa kế thừa bản chất từ `Device`, vừa bổ sung được năng lực `INetworkable`.

- Việc sử dụng interface giúp **nhóm các đối tượng theo năng lực**: Hệ thống có thể duyệt qua bất kỳ đối tượng nào (Máy tính, hay một Máy in mạng) có khả năng kết nối và gọi hàm `Connect` mà không cần biết chúng là loại máy móc gì, giúp giảm bớt sự phụ thuộc vào cây kế thừa thiết bị.

## 5.3 3. Tính đa hình được thể hiện ở đâu?

Tính đa hình được thể hiện rõ ràng nhất tại **phương thức tính toán chi phí bảo trì** trong `LabRoom`:

```
public decimal CalculateAnnualMaintenanceCost() {
    decimal totalCost = 0;
    foreach (var device in Devices) {
        totalCost += device.CalculateAnnualMaintenanceCost(); // <-- ĐA HÌNH
    }
    return totalCost;
}
```

Khi duyệt danh sách `Devices` (chứa lẫn lộn Computer, Printer, Projector), chương trình không cần dùng `if` hay `switch` để kiểm tra kiểu đối tượng. Thay vào đó, nó gọi chung hàm `CalculateAnnualMaintenanceCost()`. Tại thời điểm chạy (runtime), hệ thống sẽ tự động liên kết (dynamic binding) và gọi đúng phiên bản phương thức tính chi phí đã được ghi đè (override) trong từng lớp dẫn xuất tương ứng.

Ngoài ra đa hình còn thể hiện ở hàm `ToString()` khi hiển thị thông tin chi tiết từng đối tượng, và ở việc duyệt tập hợp các thiết bị dưới góc nhìn của giao diện `INetworkable`.